

REPORTE TAREA 2

ALGORITMOS Y COMPLEJIDAD



«¿Quién ganará? Greedys vs Dinámico vs Bruto.»

Gerardo Carreño

20 de junio de 2026

23:03

Resumen

Se implementaron y compararon cuatro algoritmos — *fuerza bruta*, *programación dinámica* y dos heurísticas *greedy* — para una variante del problema de la mochila con restricciones de tiempo y energía. Se evaluaron 27 instancias reproducibles con $n \in \{3, \dots, 200\}$, midiendo tiempo de ejecución, memoria del heap y calidad de solución respecto al óptimo. *Fuerza bruta* mostró crecimiento exponencial en tiempo (acotada a $n \leq 25$); *programación dinámica* garantizó el óptimo con costo pseudopolinomial y alto uso de memoria; ambas heurísticas *greedy* escalaron polinomialmente con uso de memoria órdenes de magnitud menor. *Greedy ratio* alcanzó consistentemente entre $\approx 80\% - 100\%$ del óptimo, con una correlación moderada respecto al total de capítulos; *greedy max* fue más rápida pero inestable, con calidad entre $\approx 24\% - 95\%$ y sin correlación significativa con ninguna variable de entrada.

1. Introducción

Los problemas de optimización de recursos no solo se presentan en la ingeniería, sino también en la vida cotidiana; un ejemplo representativo es la gestión del tiempo, un recurso escaso y de alto valor. Dado que los recursos son limitados, surge la necesidad de utilizarlos de manera eficiente, lo que da origen a los problemas de optimización. En términos generales, toda tarea u objetivo requiere de recursos para su realización, y la pregunta fundamental es cómo asignarlos de forma óptima.

Para ilustrar la problemática, consideremos el escenario de las plataformas de streaming: normalmente tienen un costo asociado para acceder a su catálogo, aunque muchas ofrecen periodos de prueba gratuitos. Si antes de que termine el periodo de prueba se plantea seleccionar los animes que generen mayor satisfacción al verlos, considerando el tiempo y la energía disponibles —bajo la restricción de evaluar únicamente capítulos completos—, esta formulación constituye una instancia del problema de la mochila, un clásico problema de optimización.

A lo largo de este informe se analizarán las distintas heurísticas existentes para resolverlo, evidenciando que algunas entregan resultados consistentemente mejores que otras. En particular, se estudiará qué parámetros determinan realmente la calidad de las soluciones, así como los costos computacionales asociados y otras técnicas de programación para la resolución del problema.

2. Entorno

Los experimentos se realizaron en un sistema con las siguientes especificaciones:

Especificaciones

- **Equipo:** LENOVO ThinkPad X13 Gen 3.
- **CPU:** Intel Core i7-1265U (4.8 GHz).
- **RAM:** 16 GB.
- **Disco:** SSD NVMe 1 TB.
- **SO:** Ubuntu 26.04 (GNOME Edition) con Linux 7.0.0-22-generic.
- **Compilador:** g++ 15.2.0 con flags `-std=c++20 -O2 -g -Wall -Wextra`.

La compilación y ejecución está automatizada con Makefile. Los tiempos se midieron en nanosegundos (*ns*) con `<chrono>` [1] y uso de memoria en kilobytes (KB) mediante el pico máximo de memoria heap asignada por cada algoritmo. [7]

2.1. Dataset

Se generaron tres conjuntos de casos de prueba. Conjunto pequeño $n \in \{3, 5, 8\}$, conjunto mediano $n \in \{20, 40, 80\}$ y un conjunto grande $n \in \{100, 150, 200\}$. Por cada n en el conjunto, se crearon 3 instancias.

De esa forma podemos evaluar el desempeño de los algoritmos en función del tamaño de la entrada, además de observar la variabilidad frente a distintos datos para un mismo n .

La lógica de generación se encuentra detallada en `implementation/scripts/testcases_generator.py`, asegurando total reproducibilidad de los conjunto de datos.

3. Resultados y Análisis

Antes de profundizar en los resultados experimentales, a continuación se presenta cómo fueron contruidos los algoritmos, detallando su lógica detrás.

Fuerza bruta

La fuerza bruta explora el espacio de soluciones de forma exhaustiva mediante *backtracking* [6]: para cada anime se prueban todas las cantidades posibles de capítulos a ver (de 0 a n_{cap_i}) y se conserva la combinación de mayor satisfacción que respete las restricciones de tiempo M y energía E .

Diseño del Algoritmo El algoritmo recorre los animes en orden. En cada llamada recursiva se itera sobre los valores $k \in \{0, 1, \dots, n_{cap_i}\}$ y, si el costo en tiempo y energía de ver k capítulos del anime i no supera los recursos restantes, se realiza la llamada recursiva con los recursos reducidos y la satisfacción acumulada. Cuando se han procesado todos los animes ($anime_i = n$) se actualiza la satisfacción global si la actual es mayor [3].

Complejidad En el peor caso, para cada anime i se exploran $n_{cap_i} + 1$ opciones. Si todos los animes tuvieran C capítulos, el árbol de recursión tendría $(C + 1)^n$ hojas, lo que da una complejidad temporal de $\mathcal{O}(C^n)$, exponencial en n . La complejidad espacial es $\mathcal{O}(n)$.

Los algoritmos greedy construyen la solución paso a paso, tomando en cada iteración el mejor candidato en ese momento, sin considerar decisiones pasadas. Para el problema se implementaron dos heurísticas con diferentes lógicas [5].

Greedy ratio

Esta heurística evalúa cada par $(anime_i, k)$ — es decir, ver exactamente k capítulos del anime i — y le asigna una *razón de eficiencia*:

$$r(i, k) = \frac{s(i, k)}{\frac{t(i, k)}{M} + \frac{e(i, k)}{E}}$$

donde s , t y e son la satisfacción, el tiempo y la energía respectivos. El denominador normaliza ambos recursos para que contribuyan proporcionalmente a las restricciones establecidas. Todos los pares se

ordenan de mayor a menor *ratio*; luego se recorren en ese orden y se selecciona el primero válido para cada anime (sin repetir la elección de un mismo anime) [4].

Complejidad La creación de pares es $\mathcal{O}(\sum_{i=1}^n n_cap_i)$. Denotando $P = \sum_i n_cap_i$, el ordenamiento cuesta $\mathcal{O}(P \log P)$ y el recorrido final es $\mathcal{O}(P)$. En total, la complejidad temporal es $\mathcal{O}(P \log P)$ y el espacio es $\mathcal{O}(P)$.

Greedy max

Esta heurística prioriza los animes con mayor satisfacción total (al ver todos sus capítulos). Los animes se ordenan de forma descendente por $s(i, n_cap_i)$ y luego se intenta incorporar cada uno completo si los recursos restantes lo permiten.

Complejidad El ordenamiento cuesta $\mathcal{O}(n \log n)$ y el recorrido es $\mathcal{O}(n)$, por lo que la complejidad total es $\mathcal{O}(n \log n)$ en tiempo y $\mathcal{O}(n)$ en espacio (para la copia del arreglo).

Limitaciones de las Heurísticas Greedy Ninguna de las dos heurísticas garantiza la solución óptima. *Greedy ratio* puede descartar una combinación de menor ratio individual que, en conjunto, aproveche mejor los recursos. *Greedy max* ignora por completo la eficiencia en recursos: un anime de alta satisfacción pero costo elevado puede bloquear varias alternativas de menor satisfacción pero mayor rendimiento combinado. En ambos casos, la naturaleza local de la decisión es un impedimento que produce soluciones subóptimas.

La programación dinámica resuelve el problema de forma exacta aprovechando su *subestructura óptima*: la solución óptima para los primeros i animes con m minutos y e unidades de energía disponibles puede construirse a partir de soluciones óptimas de subproblemas más pequeños [2] [6].

Subestructura Óptima

Sea $dp[i][m][e]$ la satisfacción máxima alcanzable considerando únicamente los primeros i animes, con a lo sumo m minutos y e unidades de energía. Para el anime i se puede elegir ver $k \in \{0, 1, \dots, n_cap_i\}$ capítulos (ver 0 capítulos equivale a no seleccionarlo). La recurrencia es:

$$dp[i][m][e] = \max_{k=0}^{n_cap_i} \left\{ dp[i-1][m - t(i, k)][e - e(i, k)] + s(i, k) \mid t(i, k) \leq m, e(i, k) \leq e \right\}$$

con los casos base $dp[0][m][e] = 0$ para todo m, e (sin animes disponibles, la satisfacción es cero).

Complejidad

La tabla tiene $(n+1)(M+1)(E+1)$ celdas. Cada celda (i, m, e) se calcula iterando sobre los n_cap_i valores de k . Denotando $C = \max_i n_cap_i$, el costo total es:

$$\mathcal{O}(n \cdot M \cdot E \cdot C)$$

en tiempo, y $\mathcal{O}(n \cdot M \cdot E)$ en espacio para almacenar la tabla tridimensional. Dado que M y E pueden ser grandes, esta complejidad es *pseudopolinomial*: polinomial en los valores enteros pero exponencial en binario. Para instancias con M y E acotados, este enfoque produce la solución óptima de forma eficiente, a diferencia de la fuerza bruta (ver [fig. 5](#)).

Los resultados de la fase experimental se analizan a continuación.

Análisis de Rendimiento Computacional

Los gráficos nos muestran que el comportamiento experimental no está tan alejado del marco teórico sobre la complejidad de cada algoritmo. Las [figs. 1](#) y [7](#) ilustran la drástica diferencia entre los enfoques. En una escala logarítmica, se observa cómo el tiempo de ejecución del algoritmo de fuerza bruta crece de manera exponencial, volviéndose poco eficiente para valores de n superiores a 25. En contraste, los demás algoritmos muestran un crecimiento polinómico, lo que los hace viables para entradas considerablemente más grandes.

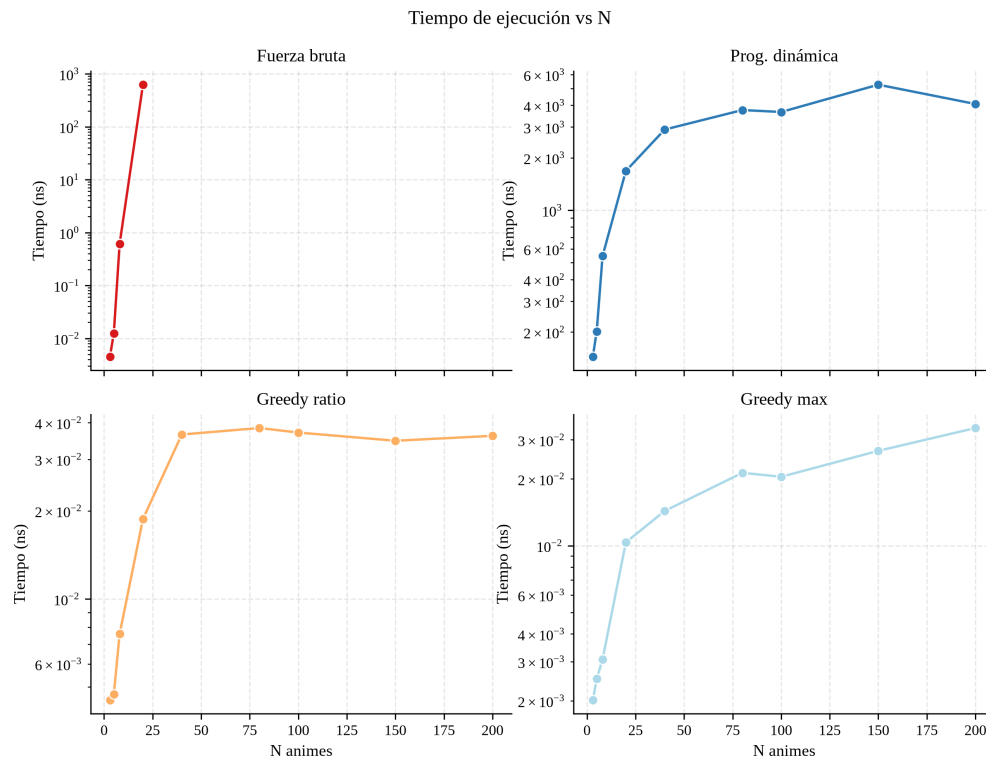


Figura 1: Comparación del tiempo de ejecución (en nanosegundos, escala *log*) en función del tamaño de la entrada n . Se evidencia el crecimiento exponencial de la fuerza bruta frente al crecimiento polinómico de los otros tres algoritmos.

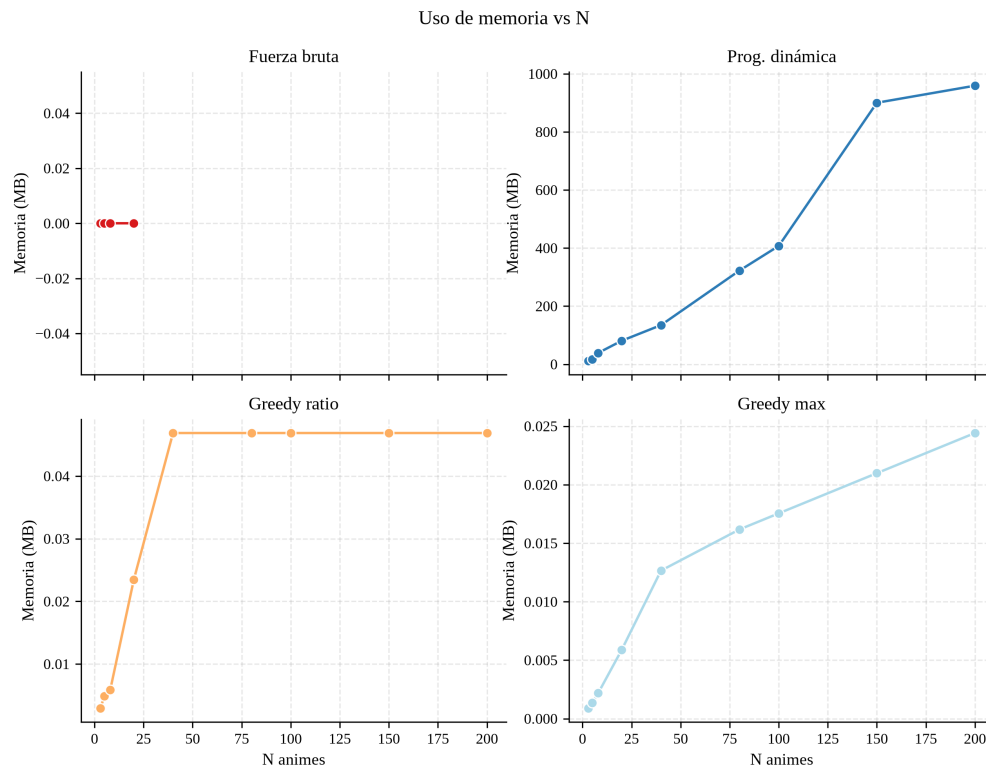


Figura 2: Comparación del uso de memoria (en KB) en función del número de anime n .

Uso de memoria La figura [fig. 2](#) muestra los resultados experimentales del uso de memoria en función de n . El enfoque de programación dinámica es el más costoso, siguiendo una curva superlineal. La razón proviene del enfoque *bottom-up*, haciendo uso de la tabulación para almacenar resultados previamente calculados. Los algoritmos de *greedy ratio* y *greedy max* ambos presentan un rápido crecimiento para pequeños valores de n , la diferencia radica en la estabilidad que alcanzan a largo plazo. *Greedy ratio* se estabiliza alrededor de $n = 40$, esto indica que no necesita mantener una estructura fija dependiendo del valor de n , mientras *greedy max* presenta una curva lineal, indicando que sí existe dependencia entre la estructura de almacenamiento y el valor de n . Para fuerza bruta, como mencionamos anteriormente, la instancia de ejecución fue acotada para valores $n \leq 25$ y el uso de memoria se observa constante a lo largo de este intervalo.

En general, los algoritmos greedy resultan considerablemente más eficientes en memoria que la programación dinámica, lo que los hace preferibles cuando el n es grande.

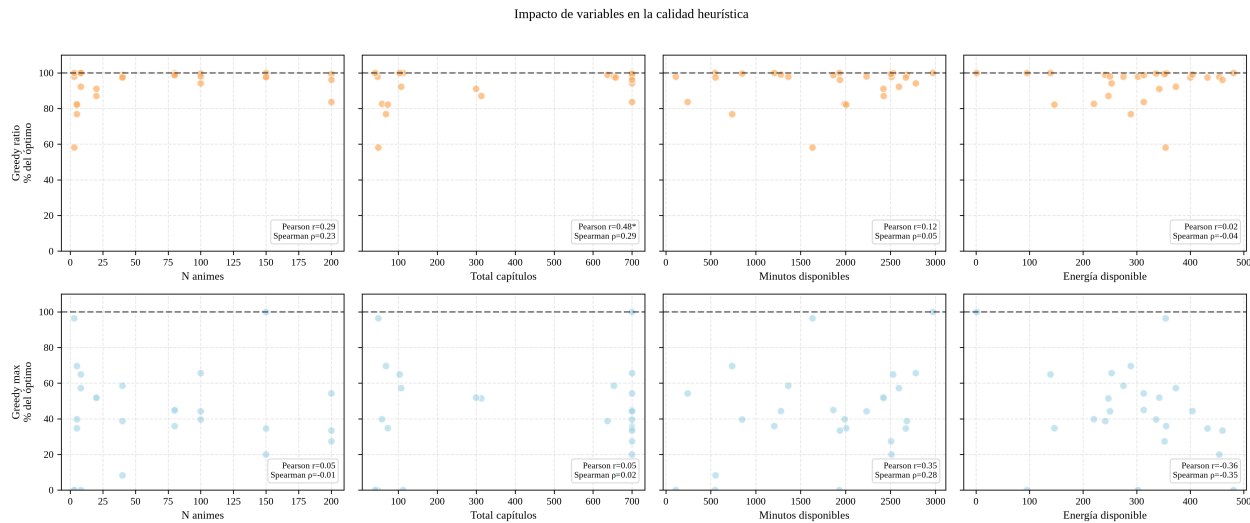


Figura 3: Comparación entre heurísticas, considerando las diferentes variables n .

Importancia de las variables en las heurística

La [fig. 3](#) nos muestra las variaciones de calidad (% del óptimo) de ambas heurísticas utilizando técnicas de correlación.

Greedy ratio no presenta una correlación relevante con N animes ($r = 0,29$), minutos disponibles ($r = 0,12$) ni energía disponible ($r = 0,02$). Sin embargo, sí se observa una correlación moderada y significativa con el total de capítulos ($r = 0,48$, $p < 0,05$). En otras palabras, ni el tamaño de la instancia ni la cantidad de recursos disponibles determinan por sí solos qué tan cerca del óptimo queda la solución, aunque la cantidad total de capítulos parece tener cierta influencia.

Greedy max no muestra correlación significativa con N animes ($r = 0,05$) ni con total de capítulos ($r = 0,05$), a diferencia de lo que podría esperarse dada la lógica de la heurística. En cambio, presenta correlaciones moderadas con minutos disponibles ($r = 0,35$) y energía disponible ($r = -0,36$), aunque ninguna alcanza significancia ($p > 0,05$ en ambos casos). Esto sugiere que la calidad de *greedy max* está más influenciada por la disponibilidad de recursos que por el tamaño de la instancia.

En definitiva, ninguna variable de entrada resulta fuertemente determinante para ninguna de las dos heurísticas, aunque *greedy ratio* muestra cierta sensibilidad al total de capítulos y *greedy max* a la disponibilidad de recursos.

Calidad de Solución

Si bien la eficiencia es punto importante, la calidad de la solución es el objetivo. Al comparar la satisfacción total obtenida por las heurísticas greedy con la solución óptima usando programación dinámica (ver [fig. 6](#) y [fig. 8](#)), se observa que la heurística *greedy ratio* de forma consistente encuentra soluciones de mayor calidad que *greedy max*, donde en muchos casos logra dar con el óptimo. Con esto queda en evidencia que la estrategia de *greedy max* tiene una lógica ingenua y con frecuencia queda atrapada en óptimos locales de baja calidad.

La relación entre el tiempo de ejecución y la calidad de la solución se observan de manera clara en la [fig. 4](#). El gráfico nos muestra el costo computacional (tiempo de ejecución) y la satisfacción obtenida (calidad de la solución como porcentaje del óptimo).

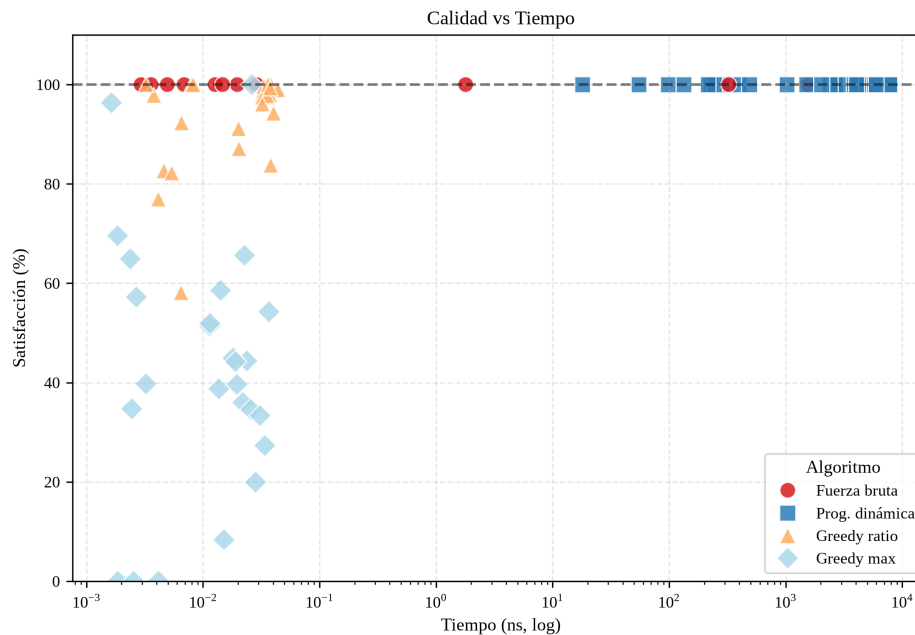


Figura 4: Trade-off entre eficiencia y calidad. La calidad de la solución expresada como porcentaje respecto al óptimo de DP.

De la [fig. 4](#) se puede concluir que:

- **Programación Dinámica (DP):** Garantiza la solución óptima (100 % de calidad) pero con el mayor tiempo de ejecución. Es la elección correcta cuando la optimalidad es un requisito estricto.
- **Greedy ratio:** Ofrece el mejor equilibrio, con una calidad muy cercana al óptimo (alrededor del 80-100 %) pero con órdenes de magnitud menor que DP respecto del tiempo de ejecución. Si nuestro problema nos permite tolerar una pequeña desviación respecto del óptimo, esta heurística es la correcta.
- **Greedy max:** Es el algoritmo más rápido, pero a costa de una calidad significativamente inferior. Es útil solo cuando la velocidad es crítica y se pueden aceptar soluciones de baja calidad.

No existe un algoritmo *mejor* que otro; la elección depende del contexto. Si la optimalidad es un requisito estricto, DP es la única opción viable. Si se necesita una solución rápida para una entrada grande y una aproximación del 80-100 % al óptimo es aceptable, *Greedy ratio* es una alternativa pragmática.

4. Conclusiones

Los resultados confirman que no existen las balas de plata, es decir que no existe un algoritmo que sea infalible, sino una relación de compromiso entre optimalidad y costo computacional. La fuerza bruta

garantiza la solución óptima pero su crecimiento exponencial la vuelve infactible para $n > 25$, mientras que la *programación dinámica* logra la misma optimalidad con un costo pseudopolinomial, a cambio de un uso de memoria considerablemente mayor debido al uso de la tabulación. Entre las heurísticas, *greedy ratio* demostró ser la opción más equilibrada, alcanzando de forma consistente — entre la mayoría de las instancias— soluciones de calidad óptima con tiempos de ejecución en órdenes de magnitud menores que la *programación dinámica*, mientras que *greedy max*, pese a ser el algoritmo más rápido y liviano en memoria, entrega soluciones de baja calidad, sin una dependencia clara respecto a ninguna de las variables de entrada. En definitiva, la elección del algoritmo debe ajustarse al contexto: la *programación dinámica* cuando la optimalidad es indispensable, y *greedy ratio* cuando se prioriza la eficiencia con una pérdida de calidad aceptable.

A. Apéndice 1

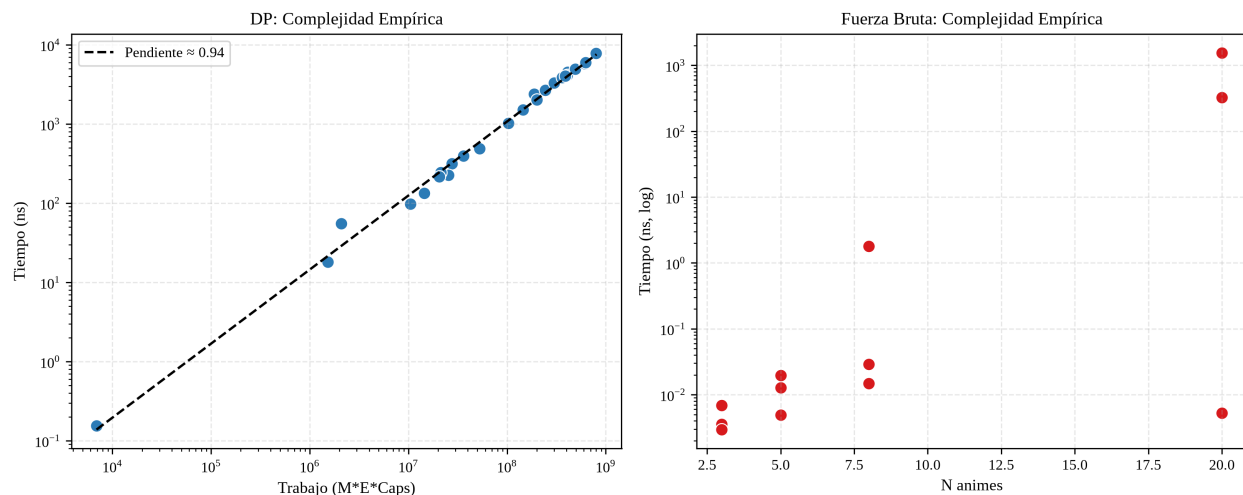


Figura 5: *Izquierda*: DP en escala log-log, pendiente $\approx 0,93$, consistente con $\mathcal{O}(n \cdot M \cdot E \cdot C)$. *Derecha*: fuerza bruta en escala log, crecimiento exponencial visible hasta $n = 20$.

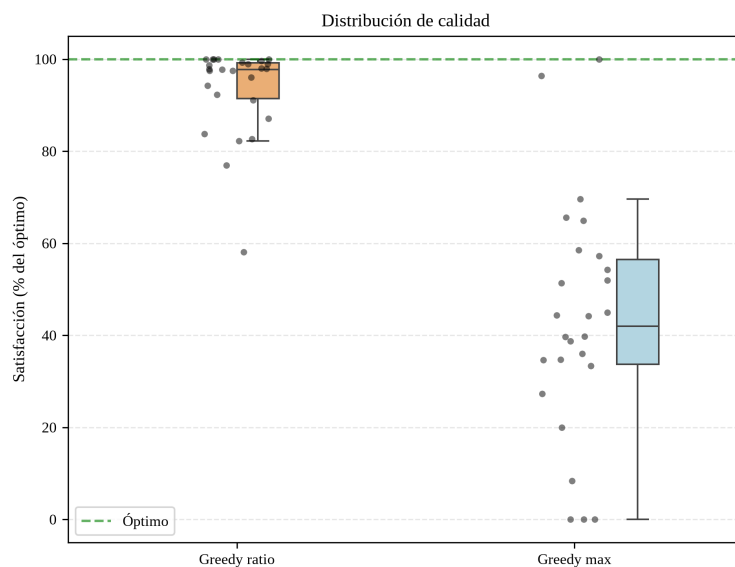


Figura 6: Calidad de solución (% del óptimo) por heurística. *Greedy ratio* se concentra sobre el 90% (mediana $\approx 95\%$); *greedy max* presenta alta dispersión (mediana $\approx 37\%$, con varios casos en 0%).

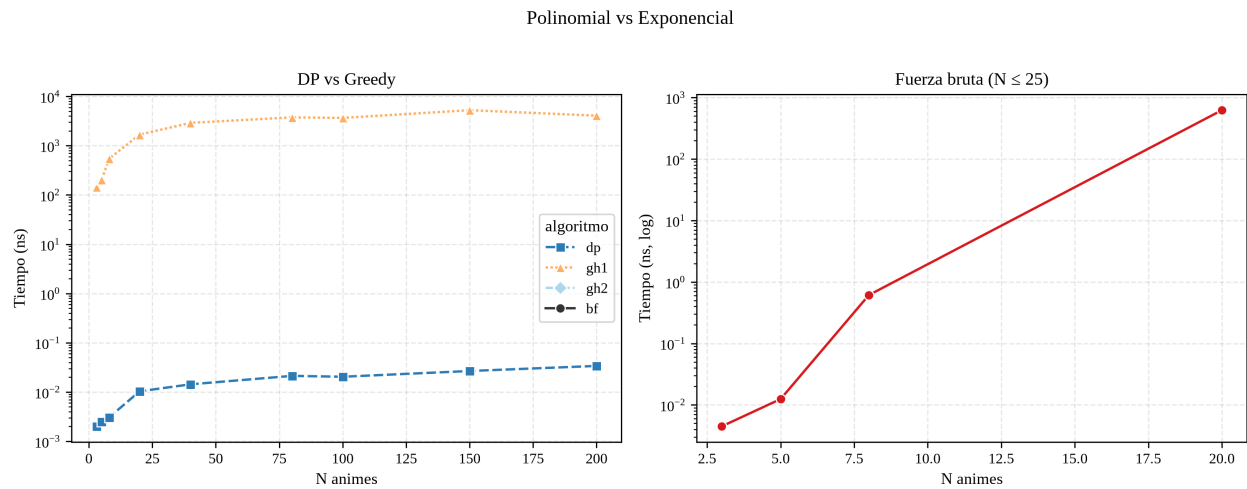


Figura 7: DP y *greedy ratio* escalan establemente hasta $n = 200$. *Fuerza bruta* crece exponencialmente, infactible para $n \geq 25$.

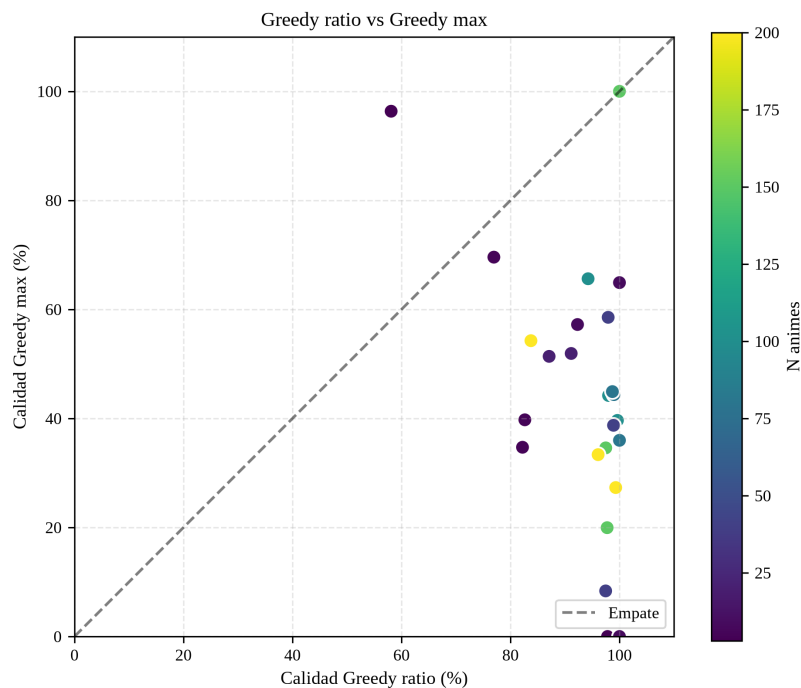


Figura 8: Calidad por instancia, *greedy ratio* vs. *greedy max*. *Greedy ratio* iguala o supera a *greedy max* en el 100% de los casos.

Referencias

- [1] cppreference.com. *Date and time utilities — std::chrono*. Accessed: 20 de junio de 2026. 2024. URL: <https://en.cppreference.com/w/cpp/chrono>.
- [2] GeeksforGeeks. *0/1 Knapsack Problem*. Accessed: 20 de junio de 2026. 2026. URL: <https://www.geeksforgeeks.org/dsa/0-1-knapsack-problem-dp-10/>.
- [3] GeeksforGeeks. *Brute Force Approach and its Pros and Cons*. Accessed: 20 de junio de 2026. 2024. URL: <https://www.geeksforgeeks.org/dsa/brute-force-approach-and-its-pros-and-cons/>.
- [4] GeeksforGeeks. *Fractional Knapsack Problem*. Accessed: 20 de junio de 2026. 2026. URL: <https://www.geeksforgeeks.org/dsa/fractional-knapsack-problem/>.
- [5] GeeksforGeeks. *Greedy Algorithms*. Accessed: 20 de junio de 2026. 2024. URL: <https://www.geeksforgeeks.org/dsa/greedy-algorithms>.
- [6] Tim Roughgarden. *Algorithms Illuminated, Parts 1-4*. Four-part series covering divide and conquer, graph algorithms, greedy algorithms, and dynamic programming. Soundlikeyourself Publishing, 2017.
- [7] Stack Overflow. *Keeping track of used memory with overloaded new and delete operator*. Accessed: 20 de junio de 2026. 2018. URL: <https://stackoverflow.com/questions/49790824/keeping-track-of-used-memory-with-overloaded-new-and-delete-operator>.